Application Management (Lmod)

Owned by Ryan Seaman
Last updated: May 05, 2025 · 2 min read · 3 people viewed

User's Tour of the Module Command

The module command sets the appropriate environment variable independent of the user's shell. Typically the system will load a default set of modules. A user can list the modules loaded by running:

```
$ module list
```

To find out what modules are available to be loaded a user can do:

```
$ module avail
```

To load packages a user simply does:

```
$ module load package1 package2 ...
```

To unload packages a user does:

```
$ module unload package1 package2 ...
```

A user might wish to change from one R version to another:

```
$ module swap R/4.2.1 R/4.4.3
```

The above command is short for:

```
$ module unload R/4.2.1
$ module load R/4.4.3
```

If a module is not available then an error message is produced:

```
$ module load packageXYZ

Warning: Failed to load: packageXYZ
```

It is possible to try to load a module with no error message if it does not exist:

```
$ module try-load packageXYZ
```

Modulefiles can contain help messages. To access a modulefile's help do:

```
$ module help packageName
```

To get a list of all the commands that module knows about do:

```
$ module help
```

The module avail command has search capabilities:

```
$ module avail cc
```

will list for any modulefile where the name contains the string "cc".

Modulefiles can have a description section know as "whatis". It is accessed by:

```
$ module whatis pmetis
pmetis/3.1   : Name: ParMETIS
pmetis/3.1   : Version: 3.1
pmetis/3.1   : Category: library, mathematics
pmetis/3.1   : Description: Parallel graph partitioning..
```

Finally, there is a keyword search tool:

```
$ module keyword word1 word2 ...
```

This will search any help or whatis description for the word(s) given on the command line.

Another way to search for modules is with the "module spider" command. This command searches the entire list of possible modules. The difference between "module avail" and "module spider" is explained in the "Module Hierarchy" and "Searching for Modules" section.

```
$ module spider
```

ml: A convenient tool

For those of you who can't type the *mdoule*, *moduel*, err *module* command correctly, Lmod has a tool for you. With **ml** you won't have to type the module command again. The two most common commands are *module list*and *module load <something>* and **ml** does both:

```
$ ml
```

means *module list*. And:

```
$ ml foo
```

means *module load foo* while:

```
$ ml -bar
```

means *module unload bar*. It won't come as a surprise that you can combine them:

```
$ ml foo -bar
```

means *module unload bar*; *module load foo*. You can do all the module commands:

```
$ ml spider
$ ml avail
$ ml show foo
```

If you ever have to load a module name *spider* you can do:

```
$ ml load spider
```

If you are ever force to type the module command instead of **ml** then that is a bug and should be reported.

Related content
R Modules
Analysis Cluster
More like this
Informational Commands
Analysis Cluster
Read with this
Python Modules
Analysis Cluster
More like this
Slurm
Analysis Cluster
Read with this
Using an Interactive Bash Shell on the Analysis Cluster
Analysis Cluster
Read with this
Partitions
Analysis Cluster
Read with this